Assignment #2 — Object Recognition and Image Retrieval

Guia completo para perceber, planear e concluir o trabalho
Advanced Computer Vision / AI for Computer Vision · 2026/27

Prazo de entrega (enunciado)	**8 de novembro de 2026** (domingo). Hora e plataforma de entrega: *não constam do enunciado*.
Hoje	22 de outubro de 2026 (quinta) → 17 dias até ao prazo
Grupo	2 pessoas (divisão proposta na secção 10)
Entregáveis	Relatório + código/notebooks + README + ficheiro de ambiente + scripts que reproduzem métricas e gráficos
Fontes usadas neste guia	Enunciado 2026/27; relatório do ano anterior (CVC 2025/26); `utils.py`, `features.py`, `encoders.py`, `train.py`, `extract.py`, `deep_models.py`, `visualization.py`; notebooks `1_handcraft.ipynb` e `2_deep_models.ipynb`; captura de ecrã da estrutura de pastas.

> **Convenção deste guia**
> Caixas roxas **"Não sei / confirmar"** marcam pontos que não consegui verificar nos materiais (por exemplo, o `README.md` do repositório não me foi dado). Não assumi nada nesses pontos — perguntem ao professor ou confirmem nos ficheiros. Caixas vermelhas são armadilhas; verdes são dicas; azuis são contexto.

Índice

1. O trabalho numa página
2. O que não sei — pontos a confirmar
3. Materiais fornecidos: pastas, dados e código
4. Conceitos essenciais (explicados do zero)
5. Métricas de avaliação
6. Protocolo experimental (regras que contam para a nota)
7. Parte a parte: o que fazer, como, e pistas para as perguntas
8. Problemas detetados no código fornecido
9. Como usar (e não usar) o relatório do ano anterior
10. Divisão de trabalho e calendário
11. Relatório final e checklist
12. Anexo: excertos de código prontos a adaptar

1. O trabalho numa página

Objetivo principal: construir e avaliar **classificadores de imagem** e **sistemas de pesquisa de imagens (retrieval)** usando três famílias de representações, e compará-las em **exatidão, eficiência com poucos dados, robustez e custo computacional**, sempre com o mesmo protocolo de avaliação:

1. **Descritores "artesanais" (handcrafted):** SIFT denso → BoVW, VLAD, Fisher Vectors → SVM linear.
2. **Representações pré-treinadas:** CNN (ResNet-18), ViT, DINOv2 (auto-supervisionado) → SVM linear sobre features congeladas.
3. **Semelhança imagem–texto:** CLIP em modo *zero-shot* (sem treino na tarefa).

Categorias: aviões (*aeroplane*), motas (*motorbike*), pessoas (*person*), cavalos (*horse*), carros (*car*) + imagens de *background* (negativos).

As 5 partes

Parte	O que é pedido	Peso de trabalho estimado
1 — BoVW + SVM linear (Stages A–G)	Classificador mota vs. background; ranking, curva PR e AP; outras classes; normalização L1/L2/nenhuma; com/sem tiling espacial; kernel de Hellinger; curva de aprendizagem (10/25/50/100% dos dados); ≥2 tamanhos de vocabulário adicionais.	Médio (descritores fornecidos para K=512)
2 — VLAD e Fisher Vectors (H–I)	Repetir classificação/retrieval com VLAD e FV; comparar AP, dimensão, tempo e armazenamento.	Baixo–médio (encodings fornecidos)
3 — Vocabulário grande + PCA vs. vocabulário pequeno	≥3 tamanhos de K; BoVW com e sem PCA (ex.: 128/256 dims); baseline pequeno com dimensão semelhante; accuracy, macro-F1, mAP, tempos, armazenamento, ≥3 seeds.	**Alto em computação** (treinar encoders novos e extrair SIFT de todas as imagens)
4 — CNN, DINOv2, CLIP (A–F)	Features congeladas de ResNet-18 (3 últimas camadas) e ViT + SVM; DINOv2 + SVM; few-shot (1/5/10/20/todos) com/sem flip; escolha de C em validação; CLIP zero-shot com vários prompts; análise visual de erros (saliência/CAM).	Alto (escrever script de extração; depois é barato)
5 — Desafio de retrieval	Escolher categoria, construir corpus + consultas + ground truth; comparar ≥2 estratégias (BoVW, VLAD/FV, CNN/DINOv2, CLIP texto→imagem); P@5/10/20, AP, mAP, R@K, tempos, exemplos de sucesso/falha.	Médio

Avaliação (rubrica sugerida no enunciado)

Critério	Peso	O que significa na prática
Análise e discussão crítica	**30%**	Interpretar resultados com evidência (gráficos, exemplos), análise de erros, limitações. **É o critério mais pesado** — não basta correr código.
Avaliação quantitativa	25%	Métricas corretas, gráficos, tabelas, **medições de custo** (tempo, dimensão, armazenamento).
Implementação e correção	20%	Pipelines corretos, código claro e reprodutível.
Qualidade do relatório e referências	15%	Organização, figuras, citações, escrita.
Desenho experimental	10%	Comparações justas, splits corretos, escolha de hiperparâmetros **só em validação**.

> **Ideia-chave para ter boa nota**
> O enunciado está escrito como um estudo comparativo: cada resultado deve ter (1) o número, (2) o custo e (3) uma explicação. Uma tabela única no fim com todos os métodos é obrigatória (secção 11). Registem tudo desde o primeiro dia num ficheiro de resultados (secção 6).

2. O que não sei — pontos a confirmar

Não sei / confirmar com o professor ou nos ficheiros

1. **Conteúdo do `README.md`** : o enunciado manda lê-lo com atenção, mas não o tenho. Pode conter instruções de instalação, comandos e regras que este guia não cobre. **Leiam-no primeiro.**
2. **Split de teste:** a pasta `data/splits` só tem ficheiros `*_train.txt` e `*_val.txt` (segundo `utils.loadSamplesTable`), e o notebook usa o `val` como **teste**. O enunciado pede treino/validação/teste. Recomendo tratar `val` como teste e criar a validação a partir do treino (secção 6), mas confirmem que é isto que o professor pretende.
3. **Encodings disponíveis para *car* e *horse***: o `extract.py` aceita essas categorias, mas o notebook só menciona motorbike/aeroplane/person. Vejam se `data/encodings/bovw_K512_C0/` tem `car_*.npz` e `horse_*.npz` ; se não, têm de os gerar com `extract.py` .
4. **Número total de imagens** e se as imagens podem pertencer a várias classes ao mesmo tempo (o nome `000007.jpg` e as categorias sugerem PASCAL VOC 2007, onde isso acontece — mas não está confirmado nos materiais).
5. **Matriz de confusão / accuracy / macro-F1**: o enunciado pede-as (Parte 3 e 4), mas o pipeline fornecido é binário (classe vs. background). Não é explícito se esperam um classificador **multiclasse de 5 classes** (e o que fazer com o background e com imagens multi-classe). Proposta na secção 7, mas confirmem.
6. **“Final 3 layers”** para extração CNN: para a ResNet-18 o notebook mostra `layer2.1` , `layer3.1` , `layer4.1` — é a interpretação mais provável. Para o **ViT** (também obrigatório: “must be extracted using ResNet-18 and ViT”) não está definido o que são as “3 últimas camadas” nem como fazer o pooling (token CLS ou média dos tokens).
7. **CLIP zero-shot com 5 categorias**: incluir ou não um prompt de “background”? O notebook do professor inclui `"a photo of a background scene"` , o enunciado fala em cinco categorias.
8. **Formato da entrega**: plataforma, hora limite, limite de páginas, língua do relatório (o do ano passado foi em português), se é zip, se o repositório conta.
9. **Conteúdo de `env.yml` , `env-win.yml` , `requirements.txt`** : não os li. O wheel fornecido (`cyvlfeat-0.7.1-cp311-cp311-win_amd64.whl`) indica **Python 3.11 em Windows 64-bit**; para Linux/macOS não sei como o professor prevê instalar o `cyvlfeat` .
10. **Tempo de computação** da extração de SIFT denso e de treino de encoders na vossa máquina: desconhecido. Uma imagem gerou ~41 771 descritores (output do notebook), por isso pode ser lento — meçam com poucas imagens primeiro.
11. **Scripts futuros**: disseram que o professor pode dar mais scripts. Organizem o vosso código para os poder integrar (secção 10).
12. Se itens marcados como **“Optional”** valem pontos extra: não indicado.

3. Materiais fornecidos: pastas, dados e código

3.1 Estrutura (da captura de ecrã)

```
ASSIGN #2/
├── code/
│   ├── src/  deep_models.py  encoders.py  extract.py  features.py
│   │         train.py  utils.py  visualization.py
│   ├── 1_handcraft.ipynb   2_deep_models.ipynb
│   ├── env-win.yml  env.yml  requirements.txt  README.md
├── data/
│   ├── encoders/    ← vocabulários / GMMs treinados (.npz)
│   ├── encodings/   ← descritores de imagem já calculados (.npz), 1 pasta por encoder
│   ├── images/      ← JPEGs
│   └── splits/      ← listas <categoria>_train.txt / <categoria>_val.txt
└── wheel/ cyvlfeat-0.7.1-cp311-cp311-win_amd64.whl
```

Os notebooks estão em `code/` e usam caminhos `../data/...`; importam `from src import ...`, por isso devem ser executados a partir de `code/`.

3.2 Dados conhecidos

- **Encodings fornecidos** (nomes usados no notebook): `bovw_K512_C0`, `vlad_K32_C80`, `fv_K32_C80`. Notação: `K` = nº de palavras/componentes; `C` = dimensão PCA aplicada **aos descritores SIFT** (128→80) **antes** do clustering; `C0` = sem PCA.
- Todos usam **tiling 2×2** (4 regiões) — o `ImageEncoder.encode` faz sempre tiling.
- Formato de cada ficheiro `data/encodings/<encoder>/<categoria>_<train|val>.npz`: chaves `file_names` (nomes sem extensão) e `histograms` (matriz N×dimensão).
- **Tarefa binária**: `loadEncodingSubset(categoria, ...)` junta positivos da categoria (rótulo +1) com **apenas** as imagens de *background* (rótulo −1). As outras classes **não** entram como negativos.

Dimensões (com 4 tiles)	Fórmula nominal	Valor
BoVW K=512	4 × K	2048 (confirmado no output do notebook)
VLAD K=32, D=80	4 × K × D	10 240 (igual ao reportado em 2025/26)
FV K=32, D=80	4 × 2 × K × D	20 480 nominal — **confirmar com `.shape`**: a função `skimage.feature.fisher_vector` pode acrescentar K termos dos pesos por tile.

Contagens	Treino (pos / neg)	"val" = teste (pos / neg)	Fonte
motorbike vs background	120 / 1019	125 / 1077	Output do notebook deste ano
aeroplane / person / horse / car	112 / 1025 / 139 / 376	126 / 983 / 148 / 337	Relatório 2025/26 — *podem ter mudado*

> **Valores de referência (para saberem se está a correr bem)**
> Notebook deste ano, BoVW K=512, L2, SVM C=10, motorbike: **AP teste = 0,5526**, 29 positivos no top-36. Relatório do ano passado: 52,07% (L2) e 66,79% (Hellinger). Valores muito diferentes destes indicam erro.

3.3 O que faz cada ficheiro

Ficheiro	Função	Quando o usam
`features.py`	Lê a imagem, normaliza para [0,1], redimensiona para altura ≤480 px, extrai **PHOW = SIFT denso em 4 escalas** (tamanhos 4,6,8,10, passo 4), descritores 128-D. Pode guardar cache por imagem.	Parte 3 e 5 (novos encoders, novas imagens)
`train.py`	Treina um encoder: amostra descritores (150 imagens/classe, 400 descritores/imagem por defeito), PCA opcional nos descritores, k-means (BoVW/VLAD) ou GMM (FV). Grava `data/encoders/<tipo>_K<k>_C<c>.npz`.	Parte 1 (K=256, 1024), Parte 3
`extract.py`	Aplica um encoder a todas as imagens e grava os encodings por categoria/subset (mesmo formato dos fornecidos).	Depois de cada `train.py`
`encoders.py`	Classes `BoVWEncoder`, `VLADEncoder`, `FisherVectorEncoder`, `ImageEncoder` (tiling 2×2), `removeSpatialInformation`.	Indiretamente; Stage E; Parte 5
`utils.py`	`train_svm` (LinearSVC, hinge), `loadEncodingSubset`, `loadSamplesTable`, `loadDataTables`, `selectTrainSamples`.	Sempre
`visualization.py`	`displayRankedImageList` (grelha 6×6), `displayPrecisionRecallCurve` (devolve AP), `RelevantVisualWordsDisplay`, `showEmbeddings2DWithLabels` (PCA + t-SNE).	Partes 1, 4, 5
`deep_models.py`	`getModel` para `resnet18`, `vit_b_32`, `dinov2_vits14`, `clip_vit_b_32` (devolve modelo em modo eval + pré-processamento oficial); `IntermediateFeatureExtractor` (guarda saídas de blocos).	Parte 4 e 5
`1_handcraft.ipynb`	Stages A–C para BoVW: carregar, normalizar, treinar SVM, ranking, curva PR, visual words. Tem o espaço para o Hellinger (`** insert code here **`).	Base da Parte 1–2
`2_deep_models.ipynb`	Demonstra ResNet-18 (camadas intermédias e CAM), sumários de DINOv2 e CLIP, e CLIP zero-shot numa imagem.	Base da Parte 4

> **Falta um script**
> Não há script para extrair features **deep** de todo o dataset. Têm de o escrever (exemplo no Anexo D) — gravem no mesmo formato `.npz` (`file_names`, `histograms`) em `data/encodings/<modelo>/` para reutilizar `loadEncodingSubset` e todo o código de avaliação da Parte 1.

3.4 Comandos (a confirmar no README)

Como `train.py` e `extract.py` usam imports relativos (`from .features import ...`), provavelmente têm de ser corridos como módulo a partir de `code/`. Os caminhos por defeito (`data/...`) não batem com a estrutura da captura (`../data/...`), por isso passem-nos explicitamente:

```
cd code
python -m src.train   -e bovw -k 256 -c 0 --data-dir ../data/splits --img-dir ../data/images \
                      --output-dir ../data/encoders --cache-dir ../cache
python -m src.extract -e ../data/encoders/bovw_K256_C0.npz --data-dir ../data/splits \
                      --img-dir ../data/images --output-dir ../data/encodings --cache-dir ../
cache
```

Dica de tempo

O `train.py` tem a cache desligada por defeito. Usem **a mesma `--cache-dir`** nos dois scripts: o SIFT de cada imagem é calculado uma vez e reutilizado por todos os encoders. Testem primeiro com `--catg motorbike` e cronometrem.

4. Conceitos essenciais (explicados do zero)

4.1 A ideia geral: de pixels a um vetor

Um classificador como a SVM precisa que cada imagem seja **um vetor de tamanho fixo**. Todo o trabalho é sobre **como construir esse vetor**. Duas abordagens:

```
HANDCRAFTED:  imagem → milhares de descritores locais SIFT (128-D cada)
              → "codificação" (BoVW / VLAD / FV) → 1 vetor por imagem → SVM linear
DEEP:         imagem → rede pré-treinada congelada → 1 embedding (128…768-D) → SVM linear
CLIP:         imagem → embedding;  texto "a photo of a horse" → embedding;  semelhança cosseno
```

4.2 SIFT denso

O SIFT descreve uma pequena região da imagem com um histograma de orientações de gradiente (4×4 células × 8 orientações = **128 números**). É robusto a mudanças de iluminação e pequenas deformações. "Denso" significa que, em vez de detetar pontos de interesse (como no Assignment 1), calcula-se o descritor **numa grelha regular** (aqui: a cada 4 píxeis, em 4 escalas). Resultado: dezenas de milhares de descritores por imagem.

4.3 Vocabulário visual e Bag of Visual Words (BoVW)

1. Junta-se uma amostra de descritores SIFT das imagens de **treino**.
2. **k-means** agrupa-os em K grupos; o centro de cada grupo é uma **"palavra visual"** (um padrão local típico: um canto, uma textura de roda, uma riscas...).
3. Para uma imagem, cada descritor é atribuído à palavra mais próxima (**quantização**) e conta-se quantos caem em cada palavra → **histograma de K bins**.

Analogia: num texto conta-se quantas vezes aparece cada palavra do dicionário, ignorando a ordem. Aqui o "dicionário" são padrões visuais.

4.4 Tiling espacial (spatial pyramid)

O BoVW perde a posição: "roda em baixo, céu em cima" fica igual a "roda em cima, céu em baixo". Dividindo a imagem em 2×2 regiões e fazendo um histograma por região, concatenados, recupera-se alguma **informação de layout**. Custa 4× a dimensão. Uma *spatial pyramid* completa concatenaria também o histograma global (nível 0); o código fornecido só faz o nível 2×2.

4.5 SVM linear

Aprende um vetor de pesos `w` e um viés `b`; a pontuação de uma imagem `x` é `s(x) = w·x + b`. Se `s>0` → positivo. Os pesos grandes indicam palavras visuais que "votam" na classe. O hiperparâmetro **C** controla o compromisso: C grande = tenta acertar todos os exemplos de treino (risco de *overfitting*); C pequeno = margem maior, mais regularização (risco de *underfitting*). Para **retrieval** usa-se `s(x)` contínuo para ordenar as imagens.

4.6 Normalização e kernels

- **L1**: divide pela soma → histograma soma 1 (distribuição de probabilidade).
- **L2**: divide pela norma euclidiana → vetor de comprimento 1. Com kernel linear, `w·x` passa a medir algo parecido com o cosseno — independente do nº de descritores da imagem.
- **Kernel linear**: semelhança $K(h,h') = \Sigma\ h_i h'_i$.

- **Kernel de Hellinger**: $K(h,h') = \Sigma \sqrt{(h_i h'_i)}$ (para histogramas L1). Truque: é um produto interno entre $\sqrt{h}$ e $\sqrt{h'}$, por isso basta transformar as features com raiz quadrada e continuar a usar a SVM linear ("feature map explícito").

4.7 VLAD e Fisher Vectors

- **VLAD**: em vez de só contar descritores por palavra, soma os **resíduos** (descritor − centro) de cada palavra. Guarda "para que lado" os descritores se desviam do centro → vetor K×D. Codificação de **1.ª ordem**.
- **Fisher Vector**: troca o k-means por um **GMM** (mistura de gaussianas) com covariâncias diagonais; cada descritor é atribuído de forma **suave** (probabilística) a todas as componentes; guarda médias (1.ª ordem) e variâncias (2.ª ordem) dos desvios → 2×K×D. Mais rico, mais caro.
- Por isso VLAD/FV usam K pequeno (32) e ainda assim têm dimensões muito maiores que BoVW.

4.8 PCA

Projeção linear que mantém as direções de maior variância. Neste trabalho aparece em **dois sítios diferentes** — não confundir:

Onde	O quê	Exemplo
Antes do clustering (já nos encoders fornecidos)	Reduz cada **descritor SIFT** 128→80	`vlad_K32_C80`, `train.py -c 80`
Parte 3 (o que é pedido)	Reduz o **vetor final da imagem** (histograma BoVW de K×4 dims) para 128/256	`sklearn.decomposition.PCA` ajustado **só no treino**

4.9 CNN, ViT, DINOv2 e CLIP

- **ResNet-18** (CNN): camadas convolucionais aprendidas em ImageNet (classificação supervisionada de 1000 classes). Cada bloco produz um mapa de features (canais × altura × largura); aplicando **Global Average Pooling** (média espacial) obtém-se um vetor: `layer2.1` →128, `layer3.1` →256, `layer4.1` →512 dims (confirmado no notebook). Camadas iniciais = bordas/texturas; finais = partes de objetos/semântica.
- **ViT-B/32**: Vision Transformer; corta a imagem em blocos 32×32, transforma-os em "tokens" e processa-os com atenção. O token especial **CLS** resume a imagem (768-D no torchvision).
- **DINOv2 ViT-S/14**: ViT treinado **sem rótulos** (auto-supervisionado: aprender a dar representações consistentes a vistas diferentes da mesma imagem). Embedding global 384-D (confirmado no notebook); também tem **tokens de patch** (16×16 = 256 tokens para entrada 224×224).
- **CLIP ViT-B/32**: treinado com pares imagem–legenda para pôr imagem e texto no **mesmo espaço** (512-D). Permite classificar comparando a imagem com frases ("a photo of a horse") sem treino: **zero-shot**.
- **Features congeladas**: a rede não é treinada; só a SVM em cima.
- **Few-shot**: treinar o classificador com muito poucos positivos (1, 5, 10…).

4.10 Classificação vs. retrieval

Classificação: decidir a classe de cada imagem (precisa de um limiar). **Retrieval**: ordenar uma coleção por relevância (só importa a **ordem**). Nas Partes 1–4 a mesma pontuação da SVM serve para as duas. Na Parte 5 não há classificador: ordena-se o corpus pela semelhança com a consulta (ex.: cosseno entre embeddings).

5. Métricas de avaliação

Métrica	Definição	Nota
Precision	TP / (TP+FP): dos que dei como positivos, quantos são	Penaliza falsos positivos
Recall	TP / (TP+FN): dos positivos que existem, quantos encontrei	Penaliza falhas
Curva Precision–Recall	Percorre o ranking do topo para baixo; em cada posição calcula (recall, precision)	Melhor que accuracy com classes desbalanceadas (~10% positivos aqui)
Average Precision (AP)	Média da precision nas posições onde aparece cada positivo	Só depende da **ordem**
mAP	Média das AP por classe (ou por consulta, na Parte 5)	Reportar também as AP por classe
Precision@K / Recall@K	Precision/recall considerando só as K primeiras imagens	Parte 5: K = 5, 10, 20
Matriz de confusão	Linhas = classe real, colunas = classe prevista	Exige classificação com decisão (ver secção 7)
Accuracy / macro-F1	% acertos / média do F1 de cada classe	Accuracy binária é enganadora com 90% negativos

Exemplo resolvido de AP

```
Ranking (do maior score para o menor):  +  −  +  +  −     (3 positivos no total)
Precision nas posições dos positivos:   1/1, 2/3, 3/4  = 1,000 ; 0,667 ; 0,750
AP = (1,000 + 0,667 + 0,750) / 3 = 0,806
Se trocar as posições 1 e 2 (− + + + −):  1/2, 2/3, 3/4 → AP = 0,639
```

> **Definam a convenção**
> O enunciado pede “define the positive class and the convention used for AP”. O código usa `sklearn.metrics.average_precision_score` : AP **não interpolada** (soma em degraus da precision × variação do recall). Escrevam isto no relatório. O relatório do ano passado também reportava AUC e “AP11” (interpolada em 11 pontos), que dão valores ligeiramente diferentes.

6. Protocolo experimental (regras que contam para a nota)

1. **Três conjuntos**: treino → ajustar modelos; validação → escolher hiperparâmetros (C, normalização, K, prompts, camada, dimensão PCA); teste → **uma vez**, no fim.
2. **Proposta** (sujeita à confirmação da secção 2): ficheiros `*_val` = **teste**; validação = 20–25% do treino, estratificada e com seed fixa (Anexo A). Depois de escolher as configurações, retreinar em todo o treino e avaliar no teste.
3. **Mesmas imagens de teste e mesmos rótulos** para todos os métodos (incluindo CLIP).
4. **Experiências com poucos dados**: ≥3 seeds, reportar média ± desvio padrão.

5. **Registar**: versões de software, identificador do modelo e pesos, pré-processamento, seeds, hiperparâmetros, hardware (CPU/GPU), e se os tempos incluem carregamento do modelo e pré-processamento.
6. **Custos**: dimensão da feature, tempo de extração, tempo de treino do classificador, tempo de inferência por imagem, armazenamento aproximado (bytes/imagem e total).

Fuga de informação (leakage) — o erro mais comum

O relatório do ano passado escolheu configurações olhando para o AP de teste. Este ano isso é explicitamente proibido. Também: o k-means/GMM e o PCA da Parte 3 têm de ser ajustados **só com treino**. Os encoders fornecidos foram treinados com imagens de treino (`train.py` usa `subset="train"`), o que inclui as imagens que vão para a vossa validação — é aceitável (não usa rótulos de forma supervisionada), mas mencionem como limitação.

Registo de resultados desde o dia 1

Cada experiência acrescenta uma linha a um `results.csv` partilhado (Anexo G) com: `parte, metodo, encoding, classe, normalizacao, kernel, C, fracao, seed, split, AP, dim, t_extracao, t_treino, t_inferencia, bytes` . No fim, as tabelas e gráficos do relatório saem deste ficheiro por script — isso também cumpre o requisito de reprodutibilidade.

7. Parte a parte: o que fazer, como, e pistas para as perguntas

"Pistas" = pontos que uma boa resposta deve cobrir; as respostas finais têm de ser apoiadas pelos **vossos** resultados.

Parte 1 — BoVW e SVM linear

Stage A — Dados e BoVW

- Mostrar imagens representativas de cada classe e do background; comentar escala, ponto de vista, iluminação, oclusão, fundo.
- Usar `bovw_K512_C0` fornecido. Gerar **pelo menos mais dois** vocabulários (ex.: K=256 e K=1024) com `train.py` + `extract.py` (mesmos splits). Estes encodings servem também para a Parte 3 — façam-nos cedo.
- Comparar histograma global vs. tiling 2×2 (ver bug na secção 8 antes de usar `removeSpatialInformation`).

Q: Como escolher K? Por validação (AP vs. K), equilibrando com custo; K pequeno junta padrões diferentes na mesma palavra; K grande dá histogramas esparsos e ruidosos com poucos dados.
Q: Tiling codifica layout? Quando prejudica? Codifica posição grosseira (céu em cima, estrada em baixo). Prejudica quando o objeto aparece em posições/escala variáveis, com flips, e porque multiplica a dimensão por 4 (mais parâmetros com o mesmo nº de exemplos).
Q: K vs. capacidade, erro de quantização, custo, armazenamento? Capacidade ↑ e erro de quantização ↓ com K; k-means custa ~proporcional a N·K·D por iteração; codificar custa K distâncias por descritor; armazenamento ∝ 4K. Meçam os tempos.

Stage B — Classificador de motas

- Já está no notebook (C=10). Mostrar top/bottom do ranking de treino e usar `RelevantVisualWordsDisplay` para as palavras com peso mais alto.

Q: O que esperam? Escrevam a previsão **antes** de correr (é o que se pede). **Q: Topo partilha objeto, contexto ou fundo?** Verifiquem se as palavras relevantes caem na mota ou na estrada/paisagem (o ano passado viu muito fundo). **Q: Treino alto, teste baixo?** Overfitting: 2048 dims com ~120 positivos; o AP de treino é otimista porque o modelo viu essas imagens.

Stage C — Teste e retrieval

- Pontuar o teste, curva PR, AP, nº de positivos no top-36.

Q: Porque é que o bias é irrelevante para a ordem? Somar a mesma constante `b` a todas as pontuações não altera a ordem; como a curva PR e o AP só dependem da ordem, não mudam. O `b` só importa para a **decisão** (limiar 0 → precision/recall num ponto, accuracy). *A explicação do ano passado (normalização) não é a razão correta.*
Q: Ordem dos positivos/negativos vs. PR e AP? Cada negativo acima de um positivo baixa a precision nesse ponto; positivos no topo → precision alta a recall baixo; AP = área sob a curva (use o exemplo da secção 5).

Stage D — Outras categorias

- Repetir para aeroplane, person, horse, car (se houver encodings — secção 2). Tabela de AP por classe + mAP; mesmo protocolo.

Q: Porque variam? Nº de exemplos (person tem ~8× mais), variação intra-classe, tamanho do objeto (aviões pequenos no céu), diversidade do fundo, co-ocorrência (pessoas aparecem em imagens de motas/cavalos). Apoiem com contagens e exemplos.

Stage E — Normalização e informação espacial

- Nenhuma vs. L1 vs. L2, escolhidas em **validação**; tiling vs. global.

Q: L2 e auto-semelhança? $K(h,h)=\|h\|^2=1$ para todas as imagens. **Q: vs. K(h,h')?** $K(h,h')=\cos\theta \leq 1$ (Cauchy–Schwarz), igual a 1 só se h'=h; ≥0 porque os histogramas são não-negativos. **Q: Sem normalização / L1?** Não: $K(h,h)=\|h\|_2^2$ varia de imagem para imagem (sem normalização depende do nº de descritores, isto é, do tamanho da imagem; com L1 fica ≤1 e só é 1 se o histograma tiver um único bin). Uma imagem pode ser "mais parecida" com outra do que consigo própria. **Q: Correlação com desempenho?** Empírica — mostrem a tabela. **Q: Remover espacial ajuda umas classes e prejudica outras?** Depende da consistência do layout por classe.

> **Pista que vale discussão**
> Com C fixo (=10), vetores L1 têm magnitudes muito pequenas (soma 1 distribuída por 2048 bins), o que equivale a regularização muito mais forte. Parte do mau resultado L1 do ano passado (26% AP) pode vir daí e não da L1 em si. Testar C em validação para cada normalização é uma análise forte.

Stage F — Hellinger

- No sítio `** insert code here **` do notebook: transformar treino e teste (Anexo B). Comparar com as outras normalizações.

Q: Como normalizar? Antes ou depois da raiz? Se h está normalizado em **L1**, então $\|\sqrt{h}\|_2^2 = \Sigma h_i = 1$: a raiz de um histograma L1 já tem norma L2 unitária. Logo "L1 e depois raiz" ≡ "raiz e depois L2". O que não funciona é L2 antes da raiz (o resultado deixa de ter norma 1).
Q: Efeito em palavras frequentes vs. raras? A raiz comprime contagens grandes (palavras muito repetidas, ex.: textura do fundo — "burstiness") e aumenta o peso relativo das raras.

Stage G — Nº de imagens de treino

- Frações 10/25/50/100% do treino, ≥3 seeds, linear e Hellinger; gráfico **AP de validação vs. nº de exemplos** com barras de erro. Usem `utils.selectTrainSamples` (tem seed) em vez do `get_subset` do notebook (sem seed).

Q: Satura? Mais dados ou mais diversidade? Leiam a inclinação da curva no fim. Se ainda sobe, mais dados ajudam; se achatou mas o fosso treino–validação é grande, diversidade/melhor representação ajuda mais.

Parte 2 — VLAD e Fisher Vectors

- Trocar `encoding` no notebook para `vlad_K32_C80` e `fv_K32_C80`; repetir classificação e retrieval para as mesmas classes. Mesmo classificador e splits.
- VLAD do `cyvlfeat` já sai normalizado em L2 por tile (`unnormalized=False`). Opcional: normalização documentada (intra-normalização por cluster + L2 global, ou raiz com sinal — Anexo B), escolhida em validação. Não usem raiz simples: VLAD/FV têm valores negativos.
- Reportar AP/mAP, dimensões (com `.shape`), tempo de extração e armazenamento.

Q: Dimensão BoVW (K) vs. VLAD (K×D)? VLAD com K=32, D=80 tem 2560 dims por tile contra 512 do BoVW K=512 — mais capacidade com menos palavras; armazenamento maior e vetores densos (float).
Q: O que FV acrescenta e a que custo? 2.ª ordem (variâncias), atribuição suave; custo: treino EM do GMM, posteriores de todos os descritores para todas as K componentes, o dobro da dimensão do VLAD.

Parte 3 — Vocabulário grande + PCA vs. vocabulário pequeno

Condição	Proposta concreta (com tiling 2×2)	Dimensão final
A — pequeno, sem PCA	BoVW K=256	1024
B — grande, sem PCA	BoVW K=1024	4096
C — grande + PCA	K=1024 → PCA 128 e 256 (e outros, para ver onde cai)	128 / 256
D — pequeno com dim. ≈ C	BoVW K=32 e K=64 (4×32=128, 4×64=256) — ou K=128/256 sem tiling	128 / 256

- Têm de treinar encoders extra (K=32, 64, 256, 1024). Usem K=512 como 3.º ponto. Declarem explicitamente se usam tiling.
- PCA: `PCA(n_components=d).fit(X_treino)`, aplicar a validação/teste; d < K×4 e d ≤ n_treino − 1. Decidam e documentem a normalização antes/depois do PCA (ex.: Hellinger → PCA → L2), escolhida em validação.
- Medir: tempo de k-means, de codificação, de ajuste do PCA, de treino SVM, inferência/imagem, armazenamento; % de dimensões removidas; ≥3 seeds (seed do k-means e/ou da divisão).
- "Where meaningful" para VLAD/FV: reduzir com PCA o FV de 20k dims é onde o PCA tem mais impacto prático — bom para responder à última pergunta.

> **Accuracy e macro-F1 — confirmar**
>
> Pedem accuracy e macro-F1 "de teste". Com um classificador binário por classe, o limiar 0 dá accuracy dominada pelos ~90% de negativos. Proposta: reportar (i) mAP sobre as tarefas classe-vs-background e (ii) uma tarefa **multiclasse 5-vias** nas imagens de teste das 5 classes: prever a classe com maior pontuação entre as 5 SVMs → accuracy, macro-F1, matriz de confusão. Imagens que pertençam a mais de uma classe têm de ter uma regra declarada (excluir ou contar acerto se prever qualquer uma). Confirmem com o professor.

Perguntas: K maior ajuda antes do PCA? Para que classes? · O PCA mantém o ganho? Até que dimensão? (gráfico AP vs. d) · À mesma dimensão, grande+PCA vence pequeno? · Trade-off exatidão/memória/tempo (distingam custo de **construir** a representação grande do custo de **guardar/classificar** a reduzida) · Diferença BoVW vs. VLAD/FV (a dimensão destes cresce com K×D).

Parte 4 — CNN, DINOv2, CLIP

Passo 0 (comum): extrair e guardar todas as features

Script único (Anexo D) que, para cada imagem de cada `categoria_subset`, grava: ResNet-18 GAP de `layer2.1/3.1/4.1`; ViT-B/32 (CLS); DINOv2 ViT-S/14 (CLS e, opcional, média dos patch tokens); CLIP imagem; e as mesmas para a imagem **espelhada** (para o Stage C). Medir tempo por imagem (GPU: `torch.cuda.synchronize()` antes de parar o relógio). A partir daí, todas as experiências são segundos.

Stage A — CNN congelada

- SVM linear em cada uma das 3 camadas da ResNet-18 + ViT; mesmos splits da Parte 1; comparar com BoVW/VLAD/FV. Normalizar L2.

Q: Aprendidas vs. handcrafted? Mais dims = melhor AP? Features aprendidas captam partes/semântica, invariâncias aprendidas em milhões de imagens; FV tem 20k dims e provavelmente perde para 512 dims da `layer4` — dimensão não é qualidade.

> **Pré-processamento corta a imagem**
> Os transforms oficiais redimensionam e fazem **center crop 224×224**: objetos nas margens podem ser cortados. Mencionem isto nas falhas.

Stage B — DINOv2

- SVM no embedding global 384-D; AP por classe, mAP, matriz de confusão (ver caixa da Parte 3), dimensão, tempo, custo de inferência. Opcional: média dos patch tokens (via `forward_features`; *confirmar o nome das chaves na versão instalada*).

Q: Porque funciona com poucos rótulos? O pré-treino auto-supervisionado em muitas imagens organiza o espaço de forma que imagens semanticamente parecidas ficam próximas → separáveis linearmente com poucos exemplos. **Q: O que se perde com 1 descritor global?** Layout espacial, objetos pequenos, múltiplos objetos.

Stage C — Few-shot e augmentation

- Uma representação (sugestão: DINOv2). Positivos: 1, 5, 10, 20, todos; negativos: **declarem** (sugestão: todos os background de treino). ≥3 seeds (mais para 1-shot, ex.: 10). Gráfico AP validação vs. nº positivos.
- Sem vs. com flip horizontal **só no treino** (adicionar as features espelhadas como exemplos extra). TTA (média original+espelhada no teste) só como condição separada.

Q: Quanto reduz a dependência de rótulos? Comparem esta curva com a do Stage G (BoVW). **Q: Quando o flip prejudica?** Quando a orientação é informativa: texto, sinais, mão esquerda/direita, objetos assimétricos com direção relevante. **Q: Média de vetores = média de scores?** Para uma SVM linear **sem** renormalização, **sim**: $w\cdot(x_1+x_2)/2 + b = (s_1+s_2)/2$. Deixa de ser exato se se renormalizar (L2) depois de fazer a média, ou com classificadores não lineares. *O relatório do ano passado diz "não" — está errado para o caso linear.*

Stage D — Escolha de C

- C ∈ {0.1, 1, 10, 100, 1000}, AP treino e validação no mesmo gráfico; escolher na validação; avaliar o escolhido uma vez no teste.

Q: Regularização e bias–variance com poucos positivos? C pequeno → mais regularização → mais bias, menos variância, protege contra overfitting com poucos positivos; C grande ajusta ao ruído. O desbalanceamento também pesa (`class_weight="balanced"` é opção, mas é uma alteração — declarar).

> **Convergência da SVM**
> `utils.train_svm` usa `max_iter=100` e esconde os avisos de não-convergência. Com C grande pode não convergir e os resultados ficam enganadores. Aumentem `max_iter` (ex.: 10 000) e imprimam os avisos.

Stage E — CLIP zero-shot

- Embeddings de imagem normalizados; um prompt por classe ("a photo of a motorbike") vs. vários templates (média dos embeddings de texto — Anexo E). Escolher prompts em validação ou por conhecimento prévio, **nunca** pelo teste.
- Classe prevista = maior cosseno → matriz de confusão e métricas por classe. Para AP por classe: ordenar o **mesmo** conjunto de teste da Parte 1 (classe + background) pelo cosseno com o prompt da classe → comparável com as SVMs.
- Recomendado: acrescentar **CLIP-imagem + SVM** como linha extra — é o que a 1.ª pergunta pede para comparar.

Q: Zero-shot vs. SVM sobre embeddings CLIP? No zero-shot os "pesos" do classificador são os embeddings de texto (sem exemplos rotulados); a SVM aprende pesos no espaço de imagem a partir de exemplos — geralmente melhor com dados suficientes, pior/igual com 1–5. **Q: Sensibilidade e risco de escolher prompts no teste?** Mostrem a variação de AP entre prompts; escolher no teste é leakage → estimativa otimista.

Stage F — Análise visual e falhas

- Exemplos certos e errados para CNN, DINOv2, CLIP.
- Mapas: o CAM do notebook usa os pesos de ImageNet (explica a classe ImageNet, não a vossa SVM). Para explicar a **vossa** SVM sobre `layer4.1`: como o score é a média espacial de `w·F(:,i,j)`, o mapa `M(i,j)=w·F(:,i,j)` é exatamente o CAM da SVM (Anexo F). DINOv2: semelhança cosseno entre patch tokens e o CLS (ou entre patches e um protótipo da classe). Qualquer modelo: **oclusão** (tapar blocos e medir a queda do score).

Q: Regiões correspondem ao objeto? O que se pode concluir? Mostram correlação com a decisão, não causalidade; baixa resolução (7×7); dependem do método. Exemplos em que o mapa cai no fundo (estrada, céu, relva) = evidência de enviesamento de contexto. **Q: Classes/condições difíceis?** Objetos pequenos, oclusão, cortados pelo crop, múltiplas classes na mesma imagem.

Parte 5 — Desafio de retrieval

1. **Categoria e corpus**: forma mais simples e limpa = usar o dataset fornecido. Ex.: categoria *horse*; **consultas** = 5–10 imagens diversas do treino de horse; **corpus** = todas as imagens de teste (todas as classes + background); relevante = imagem pertence a horse. Inclui distratores difíceis (pessoas, carros em paisagens rurais). Se recolherem imagens da web, documentem a origem e a licença.
2. **Sem duplicados** entre consultas e corpus: confirmem pelos nomes de ficheiro e procurem quase-duplicados (cosseno DINOv2 > ~0,95 entre consulta e corpus → inspecionar).
3. **Estratégias** (mínimo 2; recomendo 4, porque as features já existem): BoVW (cosseno ou intersecção de histogramas), VLAD/FV (cosseno, PCA opcional), DINOv2/CNN (cosseno), CLIP texto→imagem (várias formulações: "horse", "a photo of a horse", "a brown horse in a field"...).
4. **Métricas**: por consulta AP, P@5/10/20, R@K; mAP sobre consultas; tempos (extração, armazenamento, tempo de ordenação por consulta). Mostrar top-10 de sucesso e de falha.
5. **Sensibilidade às consultas**: grupo de consultas parecidas vs. muito diferentes; variação do AP entre consultas.

Perguntas: melhor estratégia (com números e exemplos) · local (BoVW/VLAD) vs. global deep · vocabulário maior e PCA em retrieval (reutilizem encodings da Parte 3) · diversidade das consultas · wording no CLIP · limitações de corpus pequeno (tamanho, ground truth incompleto, desbalanceamento, quase-duplicados).

8. Problemas detetados no código fornecido

1. `removeSpatialInformation` mistura palavras de tiles diferentes (bug provável)
`encode_tiling` concatena os tiles por blocos: `[tile0(K), tile1(K), tile2(K), tile3(K)]`. A função faz `reshape(N, -1, 4).mean(axis=2)`, que agrupa **4 bins consecutivos** (palavras 0–3 do tile 0, etc.) em vez do mesmo bin nos 4 tiles. Testei com um vetor de exemplo: devolve `[3.25, 16, 28.75]` quando o correto é `[15, 16, 17]`. Correção: `H.reshape(N, 4, -1).mean(axis=1)` (Anexo C). Usem a correção no Stage E e documentem-na. (Só falha o caso em que algum tile não tem pontos — o código salta tiles vazios e o vetor fica mais curto.)

2. Mesmo problema em `RelevantVisualWordsDisplay`
`w.reshape(hist.shape[0], -1)` com w de 4K dimensões agrupa pesos consecutivos. Para obter o peso médio de cada palavra nos 4 tiles: `w.reshape(4, -1).mean(axis=0)`. Além disso, a classe aplica sempre raiz + L2 ao histograma (Hellinger) mesmo que o classificador tenha sido treinado só com L2.

3. `get_subset` do notebook não tem seed
`np.random.default_rng()` sem seed → resultados não reprodutíveis. Usem `utils.selectTrainSamples(..., random_state=seed)`.

4. SVM com `max_iter=100` e avisos silenciados
(ver Parte 4, Stage D).

5. Caminhos
`utils.loadEncodingSubset` usa `root_dir="data"` por defeito; o notebook redefine-a com `"..\data"` (separador Windows). Passem `root_dir` explicitamente e usem `/` ou `pathlib` para funcionar nos dois sistemas.

Confirmem estes pontos com o professor ou mencionem-nos no relatório como “deviations from the protocol” — o enunciado pede que identifiquem desvios.

9. Como usar (e não usar) o relatório do ano anterior

Útil: estrutura das respostas da Parte 1–2, ordem de grandeza dos resultados (BoVW L2 motorbike ~52% AP teste; Hellinger ~67%; VLAD ~76%; FV ~77–78%; CNN fc7 >90%), tipo de figuras (ranking, PR, visual words, saliência).

O enunciado mudou bastante:

2025/26	2026/27 (o vosso)
Parte 3: CNN VGG (conv1…fc7)	Parte 3 nova: vocabulário grande + PCA vs. pequeno
Parte 4: imagens do Bing para treino	Parte 4: ResNet-18, ViT, DINOv2, CLIP zero-shot, análise de falhas
—	Parte 5 nova: sistema de retrieval aberto
Sem validação, sem seeds, sem custos	Validação obrigatória, ≥3 seeds, tempos/dimensão/armazenamento, tabela comparativa

Não copiar — erros e fragilidades identificados

- Bias (Stage C): a justificação dada (normalização) não é a correta — é a invariância da ordem a uma constante.
- Média de vetores vs. média de scores: responderam “não”; para SVM linear sem renormalização é igual.
- Stage G: interpretam queda <1% de 50%→100% como “overfitting”; os próprios números mostram ~6,4 pontos de diferença (45,6% vs. 52,1%) — conclusão inconsistente.
- Hiperparâmetros e normalizações escolhidos olhando para o teste (proibido este ano).
- Tabela 13 com intervalos “+2–5%” sem valores medidos; “AP Treino = 100%” não discutido como sinal de overfitting.
- Afirmações gerais sem evidência (ex.: “pessoas são invariantes a rotação”).

10. Divisão de trabalho e calendário

A = Pessoa A (“handcrafted”) · B = Pessoa B (“deep”) · A+B = ambos

Bloco	A Handcrafted	B Deep	A+B
Arranque Qui 22 – Sex 23 out	Instalar ambiente; correr `1_handcraft.ipynb` e reproduzir AP≈0,55. Escrever módulo comum: splits treino/val/teste com seed, métricas (AP, PR, P@K, R@K, confusão, macro-F1), cronómetros, `results.csv`. **Lançar** extração SIFT com cache e treino de encoders K=32, 64, 256, 1024.	Instalar ambiente (GPU se houver); correr `2_deep_models.ipynb`. Escrever e **lançar** o script de extração deep (ResNet 3 camadas, ViT, DINOv2, CLIP, + flip) para todas as imagens.	Ler o README; enviar ao professor as dúvidas da secção 2; repositório Git (sem dados/pesos no Git); combinar formato de resultados.
Bloco 1 Sáb 24 – Seg 26	Parte 1: Stages A–F (todas as classes, normalizações escolhidas em validação, tiling corrigido, Hellinger).	Parte 4 Stages A–B (ResNet/ViT/DINOv2 + SVM, AP por classe, mAP, confusão).	Revisão cruzada do módulo comum.
Bloco 2 Ter 27 – Qui 29	Stage G (frações × seeds × 2 kernels); Parte 2 (VLAD, FV, custos).	Stages C (few-shot × flip × seeds) e D (C); Stage E (CLIP, prompts em validação, CLIP+SVM).	Primeira versão da tabela comparativa a partir do CSV.
Bloco 3 Sex 30 out – Dom 1 nov	Parte 3 completa (condições A–D, PCA, ≥3 seeds, custos, gráfico AP vs. dimensão).	Stage F (CAM da SVM, patch similarity DINOv2, oclusão, galeria de falhas).	Definir corpus, consultas e ground truth da Parte 5.
Bloco 4 Seg 2 – Ter 3 nov	Parte 5: BoVW e VLAD/FV (+ efeito de K/PCA em retrieval).	Parte 5: DINOv2/CNN e CLIP texto→imagem (wording).	Métricas e figuras comuns da Parte 5; fim das ~1,5 semanas.
Relatório + revisão Qua 4 – Sex 6 nov	Escrever Métodos/Resultados/Discussão das Partes 1–3 e 5 (A).	Escrever Métodos/Resultados/Discussão da Parte 4 e 5 (B).	Introdução, tabela final, Discussão transversal, Conclusões, Referências. Cada um revê as secções do outro. Melhorias onde a análise estiver fraca.
Verificação Sáb 7 nov	Clonar o repositório numa pasta limpa e correr os scripts do README do zero; confirmar que cada número do relatório sai do `results.csv`.		Lista de desvios ao protocolo.
Entrega Dom 8 nov	Entrega (hora e plataforma a confirmar).		

Porque esta divisão

- Cada pessoa domina um pipeline completo (instalação, código, resultados, texto) — menos dependências cruzadas e menos conflitos no Git.
- Equilíbrio: A tem a computação mais pesada (Parte 3 e encoders novos), B tem mais stages mas baratos depois da extração. Ambos lançam o trabalho pesado no primeiro dia, para correr enquanto fazem o resto.
- Pontos de integração fixos: módulo de avaliação comum (dia 1), formato `.npz` comum (permite usar o mesmo código de SVM/AP para tudo), corpus da Parte 5 e tabela final.
- Se o professor der scripts novos: integrem em `src/` e mantenham o formato de saída — o resto continua a funcionar.

Riscos e plano B

- **Instalação do cyvlfeat** falha num sistema → quem tem Windows + Python 3.11 corre a extração SIFT.
- **Sem GPU** → modelos pequenos já escolhidos (ResNet-18, ViT-S/14, ViT-B/32) correm em CPU; extrair uma vez e guardar.
- **Parte 3 demasiado lenta** → reduzir `--per-class` / `--per-image` no `train.py` e declarar.
- **Atrasos** → prioridade: tabela comparativa obrigatória e análise > extras opcionais (patch tokens, TTA).

11. Relatório final e checklist

Estrutura exigida

1. Título, autores, frase com a tarefa.
2. Introdução e background (representações e classificadores).
3. Métodos e implementação: splits, pré-processamento, hiperparâmetros.
4. Resultados: tabela consolidada + gráficos obrigatórios.
5. Discussão: curvas de aprendizagem, erros, sensibilidade a prompts, custo, limitações.
6. Conclusões: melhores compromissos e evidência.
7. Referências: modelos, artigos, dataset, software e código reutilizado (os 5 artigos + 2 repositórios do enunciado).

Tabela comparativa obrigatória

Método	Supervisão	AP / mAP	Dim. feature	Custo extração	Custo inferência
BoVW + SVM	Rótulos da tarefa				
VLAD + SVM	Rótulos da tarefa				
Fisher Vector + SVM	Rótulos da tarefa				
CNN/ViT + SVM	Pré-treino + rótulos				
DINOv2 + SVM	Auto-supervisionado + rótulos				
CLIP zero-shot	Imagem–texto; sem rótulos				

Checklist antes de entregar

- ☐ Teste usado uma única vez por configuração final; tudo o resto escolhido em validação (dito explicitamente).
- ☐ Mesmo conjunto de teste para todos os métodos, incluindo CLIP.
- ☐ Seeds (≥3) e média ± desvio nos gráficos de poucos dados (Stage G, Parte 3, Stage C).
- ☐ Todas as perguntas “Q:” respondidas com referência a uma figura/tabela.
- ☐ Curvas PR, AP por classe e mAP, matriz de confusão, P/R num limiar declarado.
- ☐ Dimensões, tempos (com hardware e o que inclui), armazenamento.
- ☐ Versões de software, modelos e pesos, pré-processamento.
- ☐ README com instruções, ficheiro de ambiente, scripts que geram as métricas e figuras; sem pesos grandes nem dados não redistribuíveis.
- ☐ Lista de experiências concluídas e desvios (ex.: correção de `removeSpatialInformation`, interpretação do split).
- ☐ Referências e créditos do código do professor.

12. Anexo: excertos de código prontos a adaptar

Escritos a partir das assinaturas das funções fornecidas; não foram executados no vosso ambiente. Testem com poucas imagens primeiro.

A — Splits treino / validação / teste e avaliação

```
import numpy as np, time
from sklearn.model_selection import train_test_split
from sklearn.metrics import average_precision_score
from src import utils

ROOT = "../data"
def l2norm(X): return X / (np.linalg.norm(X, axis=1, keepdims=True) + 1e-10)

def load_splits(cat, enc, seed=0, val_size=0.25):
    X, y, n = utils.loadEncodingSubset(cat, enc, "train", root_dir=ROOT)
    Xte, yte, nte = utils.loadEncodingSubset(cat, enc, "val", root_dir=ROOT)  # usado como TESTE
    Xtr, Xva, ytr, yva, ntr, nva = train_test_split(
        X, y, n, test_size=val_size, stratify=y, random_state=seed)
    return (Xtr, ytr, ntr), (Xva, yva, nva), (Xte, yte, nte)

def fit_eval(Xtr, ytr, Xev, yev, C=10, seed=0):
    t0 = time.perf_counter(); W, b = utils.train_svm(Xtr, ytr, C=C, max_iter=10000, seed=seed)
    t_train = time.perf_counter() - t0
    t0 = time.perf_counter(); s = Xev @ W + b; t_inf = (time.perf_counter() - t0) / len(yev)
    return average_precision_score(yev > 0, s), s, t_train, t_inf
```

B — Hellinger e raiz com sinal

```
def hellinger(X):                # só para histogramas não-negativos (BoVW)
    X = X / (X.sum(axis=1, keepdims=True) + 1e-10)   # L1
    return np.sqrt(X)                                  # norma L2 fica = 1 automaticamente

def signed_sqrt_l2(X):           # VLAD/FV (têm valores negativos)
    return l2norm(np.sign(X) * np.sqrt(np.abs(X)))
```

C — Remover informação espacial (versão corrigida) e pesos por palavra

```
def remove_spatial_fixed(H, n_tiles=4):
    return H.reshape(H.shape[0], n_tiles, -1).mean(axis=1)   # média do mesmo bin nos 4 tiles

w_per_word = W.reshape(4, -1).mean(axis=0)   # substitui w.reshape(hist.shape[0], -1) na
visualização
```

D — Extração de features deep para todo o dataset (mesmo formato .npz)

```
import torch, numpy as np, time
from pathlib import Path
from PIL import Image, ImageOps
from src.deep_models import getModel, IntermediateFeatureExtractor
from src.utils import loadDataTables

df = loadDataTables("../data/splits", "../data/images")
device = "cuda" if torch.cuda.is_available() else "cpu"
model, prep = getModel("dinov2_vits14", device=device)    # ou "vit_b_32", "clip_vit_b_32"

@torch.inference_mode()
def embed(paths, flip=False, bs=32):
    out = []
    for i in range(0, len(paths), bs):
        ims = [Image.open(p).convert("RGB") for p in paths[i:i+bs]]
        if flip: ims = [ImageOps.mirror(im) for im in ims]
        x = torch.stack([prep(im) for im in ims]).to(device)
        f = model(x)                          # CLIP: model.encode_image(x)
        out.append(f.float().cpu().numpy())
    return np.concatenate(out)

name = "dinov2_vits14"                        # acrescentar "_flip" para a versão espelhada
outdir = Path(f"../data/encodings/{name}"); outdir.mkdir(parents=True, exist_ok=True)
for (cat, subset), rows in df.groupby(["category", "subset"]):
    paths = list(rows.filepath)
    t0 = time.perf_counter(); F = embed(paths)
    if device == "cuda": torch.cuda.synchronize()
    print(cat, subset, F.shape, f"{(time.perf_counter()-t0)/len(paths)*1000:.1f} ms/img")
    np.savez_compressed(outdir / f"{cat}_{subset}.npz",
                        file_names=np.array([p.stem for p in paths]), histograms=F)

# ResNet-18, 3 últimas camadas (GAP):
#   resnet, prep = getModel("resnet18", device, nofc=True); ext =
IntermediateFeatureExtractor(resnet)
#   _, feats = ext(x); g = {k: feats[k].mean(dim=(2, 3)) for k in ("layer2.1", "layer3.1",
"layer4.1")}
#   (gravar uma pasta de encodings por camada)
```

E — CLIP zero-shot com conjunto de templates

```
import clip
clip_model, _ = getModel("clip_vit_b_32", device=device)
classes   = ["aeroplane", "car", "horse", "motorbike", "person"]
templates = ["a photo of a {}.", "a photo of the {}.", "a low resolution photo of a {}."]

@torch.inference_mode()
def text_classifier(names, templates):
    Ws = []
    for c in names:
        e = clip_model.encode_text(clip.tokenize([t.format(c) for t in
templates]).to(device)).float()
        e = e / e.norm(dim=-1, keepdim=True); e = e.mean(0)
        Ws.append(e / e.norm())
    return torch.stack(Ws).cpu().numpy()            # (5, 512)

T = text_classifier(classes, templates)
S = l2norm(X_clip_test) @ T.T                       # (N, 5) cossenos
pred = S.argmax(axis=1)                             # para a matriz de confusão
# AP da classe c no MESMO teste da Parte 1 (classe c + background):
# average_precision_score(y_c > 0, l2norm(X_c) @ T[c])
```

F — CAM da vossa SVM sobre layer4.1 da ResNet-18

```
# W: pesos da SVM treinada em GAP(layer4.1) (512-D, sem PCA)
_, feats = ext(prep(img).unsqueeze(0).to(device))
F = feats["layer4.1"][0]                                    # (512, 7, 7)
M = torch.einsum("c,chw->hw", torch.as_tensor(W, dtype=F.dtype, device=F.device), F)
# score = M.mean() + b (sem normalização); com L2 o mapa só muda de escala.
# Redimensionar M para o tamanho da imagem (F.interpolate) e sobrepor, como no notebook.
```

G — Métricas de retrieval e registo de resultados

```
def retrieval_metrics(q, Corpus, rel, ks=(5, 10, 20)):    # q e Corpus já normalizados L2
    s = Corpus @ q; r = rel[np.argsort(-s)]
    out = {"AP": average_precision_score(rel, s)}
    for k in ks:
        out[f"P@{k}"] = r[:k].mean(); out[f"R@{k}"] = r[:k].sum() / rel.sum()
    return out

import csv, os
FIELDS = ["parte","metodo","encoding","classe","norm","kernel","C","fracao","seed",
          "split","AP","dim","t_extr","t_treino","t_inf","bytes_img"]
def log_result(row, path="results.csv"):
    new = not os.path.exists(path)
    with open(path, "a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=FIELDS, extrasaction="ignore")
        if new: w.writeheader()
        w.writerow(row)
```

Referências do enunciado: Lowe (2004) SIFT · Jégou et al. (2010) VLAD · Perronnin, Sánchez & Mensink (2010) Fisher Vectors · Oquab et al. (2023) DINOv2, arXiv:2304.07193 · Radford et al. (2021) CLIP, arXiv:2103.00020 · github.com/facebookresearch/dinov2 · github.com/openai/CLIP.